

VPC Peering Exercise — stockapp-vpc ↔ peer-vpc

Project: `my-generic-project-504019` **Date:** 2026-08-14 / 2026-08-15 **Goal:** Learn how VPC Peering works in GCP by connecting two VPCs and proving connectivity with a ping, SSH hop, and file transfer — at effectively zero cost. **Result:** ☒ Successful. Peering active both directions, ping succeeded, SSH hop succeeded, file transferred.

Setup

Item	Value
VPC 1	<code>stockapp-vpc</code> (holds <code>stockapp-vm</code> , has external IP <code>136.65.201.133</code>)
VPC 2	<code>peer-vpc</code> (new, created for this exercise)
Subnet (peer-vpc)	<code>peer-subnet</code> , <code>10.20.0.0/24</code> , <code>us-central1</code>
VM in peer-vpc	<code>peer-vm</code> , <code>e2-micro</code> , <code>us-central1-a</code> , no external IP (<code>--no-address</code>)
Peering connections	<code>stockapp-to-peer</code> and <code>peer-to-stockapp</code> (peering requires both directions)

Why no external IP on `peer-vm`: it didn't need internet access for this exercise, and skipping the external IP is one less thing exposed to the internet — fewer things to monitor, nothing extra to pay for.

Why same zone (`us-central1-a`) for both VMs: traffic between VMs in the same zone is free. Cross-zone (same region) is a small per-GB charge; cross-region is higher. Same-zone kept this exercise at effectively \$0 beyond the second `e2-micro` itself (which isn't free-tier eligible, but is a trivial cost against a \$300 credit).

Steps Performed

1. Created the second VPC + subnet

```
gcloud compute networks create peer-vpc --project=my-generic-project-504019 --
subnet-mode=custom

gcloud compute networks subnets create peer-subnet \
  --project=my-generic-project-504019 \
  --network=peer-vpc --region=us-central1 --range=10.20.0.0/24
```

Used a non-overlapping CIDR (`10.20.0.0/24`) since peered VPCs can't have overlapping IP ranges with `stockapp-subnet` (`10.10.0.0/24`).

2. Created the peering (both directions — peering is not transitive/automatic)

```
gcloud compute networks peerings create stockapp-to-peer \
  --project=my-generic-project-504019 --network=stockapp-vpc --peer-network=peer-
```

```
vpc
```

```
gcloud compute networks peerings create peer-to-stockapp \
  --project=my-generic-project-504019 --network=peer-vpc --peer-network=stockapp-
  vpc
```

Verified both showed **STATE: ACTIVE** via `gcloud compute networks peerings list`.

3. Created firewall rules (peering only opens the route — each VPC's firewall still governs what's allowed)

```
# Allow traffic from peer-vpc into stockapp-vpc
gcloud compute firewall-rules create allow-from-peer-vpc \
  --network=stockapp-vpc --direction=INGRESS --action=ALLOW \
  --rules=tcp,udp,icmp --source-ranges=10.20.0.0/24

# Allow SSH from stockapp-vpc into peer-vpc
gcloud compute firewall-rules create allow-ssh-from-stockapp \
  --network=peer-vpc --direction=INGRESS --action=ALLOW \
  --rules=tcp:22 --source-ranges=10.10.0.0/24

# Allow ICMP (ping) from stockapp-vpc into peer-vpc
gcloud compute firewall-rules create allow-icmp-from-stockapp \
  --network=peer-vpc --direction=INGRESS --action=ALLOW \
  --rules=icmp --source-ranges=10.10.0.0/24
```

4. Created **peer-vm** with no external IP

```
gcloud compute instances create peer-vm \
  --project=my-generic-project-504019 --zone=us-central1-a \
  --machine-type=e2-micro --network=peer-vpc --subnet=peer-subnet --no-address
```

5. Verified connectivity

```
# From stockapp-vm:
ping -c 4 10.20.0.2      # 4/4 replies, sub-millisecond – peering + ICMP rule
confirmed
```

6. SSH hop from stockapp-vm to peer-vm, then file transfer

```
# from stockapp-vm, using the key gcloud generated at ~/.ssh/google_compute_engine
ssh -i ~/.ssh/google_compute_engine 10.20.0.2

# on peer-vm:
```

```
echo "hello from peer-vm across the peering" > /tmp/peering-test.txt
exit

# back on stockapp-vm:
scp -i ~/.ssh/google_compute_engine 10.20.0.2:/tmp/peering-test.txt ~/peering-
test.txt
cat ~/peering-test.txt    # confirmed message printed
```

Troubleshooting Journey (the actual learning)

Things rarely work on the first try — here's what broke and why, in the order encountered.

1. SSH to `stockapp-vm` itself timed out

Cause: the firewall rule `stockapp-allow-ssh` was locked to a specific home IP (`/32`) from earlier in the project, and that IP had changed (common with home ISPs). **Fix:** `curl -4 -s ifconfig.me` to get the current IPv4 (note: plain `curl ifconfig.me` returned an **IPv6** address by default, which didn't match — had to force `-4`), then updated the firewall rule's `--source-ranges` to the new IP.

2. Ping to `peer-vm` timed out initially

Cause: no ICMP-specific rule existed yet on `peer-vpc` (only SSH was allowed in). **Fix:** added `allow-icmp-from-stockapp`. Also caught a syntax error along the way — ICMP has no ports, so `--rules=icmp` is correct, not `--rules=tcp:icmp`.

3. `gcloud compute ssh peer-vm --internal-ip` (from inside `stockapp-vm`) failed with an insufficient scopes error

Cause: `stockapp-vm`'s default service account doesn't have the scope to modify metadata on *other* instances (a reasonable default — VMs shouldn't be able to freely reconfigure other VMs). This is why `gcloud compute ssh` couldn't auto-push its generated key to `peer-vm`. **Fix:** the command still generated a valid keypair locally on `stockapp-vm` (`~/.ssh/google_compute_engine`). Copied the public key and manually ran `gcloud compute instances add-metadata` **from the local machine** (which has full project permissions) to authorize that key on `peer-vm`.

4. Plain `ssh 10.20.0.2` still returned "Permission denied (publickey)" even after the metadata was correctly added

Cause: plain `ssh` only tries default-named identity files (`id_rsa`, `id_ecdsa`, `id_ed25519`, etc.). The key `gcloud` generated lives at the non-default path `~/.ssh/google_compute_engine`, so it was never offered during authentication — confirmed via `ssh -v` showing every default key attempted and failing, none of them the right one. **Fix:** `ssh -i ~/.ssh/google_compute_engine 10.20.0.2` — explicitly pointing to the correct key. This is what finally worked.

5. One red herring along the way

An earlier `ssh -v 10.20.0.2` attempt returned "Connection timed out" (not "Permission denied") — this turned out to be because that attempt was run from the **local Windows machine**, not from inside `stockapp-`

vm. Once confirmed to be running from the correct host, the real behavior (TCP connects fine, host key matches, then key auth fails) came through.

Key Lessons

- **Peering $\neq$ open access.** Peering only makes a network *path* available; each VPC's own firewall rules still fully control what traffic is allowed through it. Both sides need rules for two-way traffic.
- **Peering must be created from both sides** — it's two separate peering resources, one per VPC, not one shared object.
- **IPv4 vs IPv6 matters for firewall source ranges.** `curl ifconfig.me` can silently return IPv6; use `curl -4` when the firewall rule is an IPv4 `/32`.
- **Service account scopes limit cross-VM operations.** A VM's default service account generally can't modify another instance's metadata — this is a safety boundary, not a bug, and the workaround is doing that step from an authenticated local/admin session instead.
- **gcloud compute ssh vs plain ssh are not equivalent.** `gcloud compute ssh` manages key generation, metadata push, and IAP tunneling automatically; plain `ssh` requires the right `-i <key>` path and doesn't know about any of that.
- **Same-zone placement kept this free.** No external IP on `peer-vm`, no static IP reservation, no cross-zone/region data transfer — the only real cost was the second `e2-micro` instance itself, negligible against a \$300 credit.

Cleanup (if this environment isn't needed anymore)

```
gcloud compute instances delete peer-vm --project=my-generic-project-504019 --zone=us-central1-a
gcloud compute firewall-rules delete allow-from-peer-vpc allow-ssh-from-stockapp allow-icmp-from-stockapp --project=my-generic-project-504019
gcloud compute networks peerings delete stockapp-to-peer --project=my-generic-project-504019 --network=stockapp-vpc
gcloud compute networks peerings delete peer-to-stockapp --project=my-generic-project-504019 --network=peer-vpc
gcloud compute networks subnets delete peer-subnet --project=my-generic-project-504019 --region=us-central1
gcloud compute networks delete peer-vpc --project=my-generic-project-504019
```

```
gcloud compute ssh stockapp-vm --project=my-generic-project-504019 --zone=us-central1-a gcloud
compute ssh stockapp-vm --project=my-generic-project-504019 --zone=us-central1-a ssh -i
~/ssh/google_compute_engine 10.20.0.2
```